Sprint 2 Task 1: LangGraph Structure — Proposal for Khalil

Sprint 2 — LangGraph Structure Proposal

Task 1: GraphState + Graph Builder

Owner: Khalil + Zeineb
Estimate: 4 hours
Status: Design complete, ready to implement

---

1. Overview

What We're Building

The LangGraph structure is the **brain** of the entire system. It orchestrates all agents in a state machine:

Planner → Researcher → Extractor → FactChecker → Reasoning → Teacher

Why This Matters

Without this structure, the agents would be disconnected functions. With LangGraph, we get:

- Shared state between agents
- Automatic retry logic
- Parallel execution (Extractor runs per source)
- Streaming to Streamlit
- Checkpointing (pause/resume)

---

2. LangGraph Concepts

State (GraphState)

A dictionary that all agents can read and write to. Think of it as a shared whiteboard.

Nodes

Each agent is a node (function). Each node receives the current state, does its work, and returns updates.

Edges

Connections between nodes. They define the flow.

Conditional Edges

Branching logic. For example: "If no contradictions found, skip the FactChecker."

Send() for Parallel Execution

Allows multiple instances of the same node to run simultaneously (one per source).

---

3. GraphState Design

File: src/agents/state.py

python

```
"""
LangGraph state definition for the agent pipeline.
This TypedDict is the shared state passed between all agents.
"""
from typing import TypedDict, Annotated, Optional
import operator
from src.schemas.session import SessionSchema
from src.schemas.source import SourceSchema
from src.schemas.claim import ClaimSchema
from src.schemas.common import UserLevel, SessionStatus
class GraphState(TypedDict):
    """
    Shared state for the LangGraph pipeline.
    All agents read from and write to this state.
    """

 # --- User input ---
    query: str
```

user_level: UserLevel

 # --- Pipeline control ---

session_id: str

status: SessionStatus

current_agent: str

retry_count: int

 # --- Planner output ---

sub_queries: list[str]

 # --- Researcher output ---

sources: list[SourceSchema]

 # --- Extractor output (parallel accumulation) ---

claims: Annotated[list[ClaimSchema], operator.add]

 # --- FactChecker output ---

contradictions: list[dict]

has_contradictions: bool

 # --- Reasoning output ---

reasoning: str

 # --- Teacher output ---

final_response: str

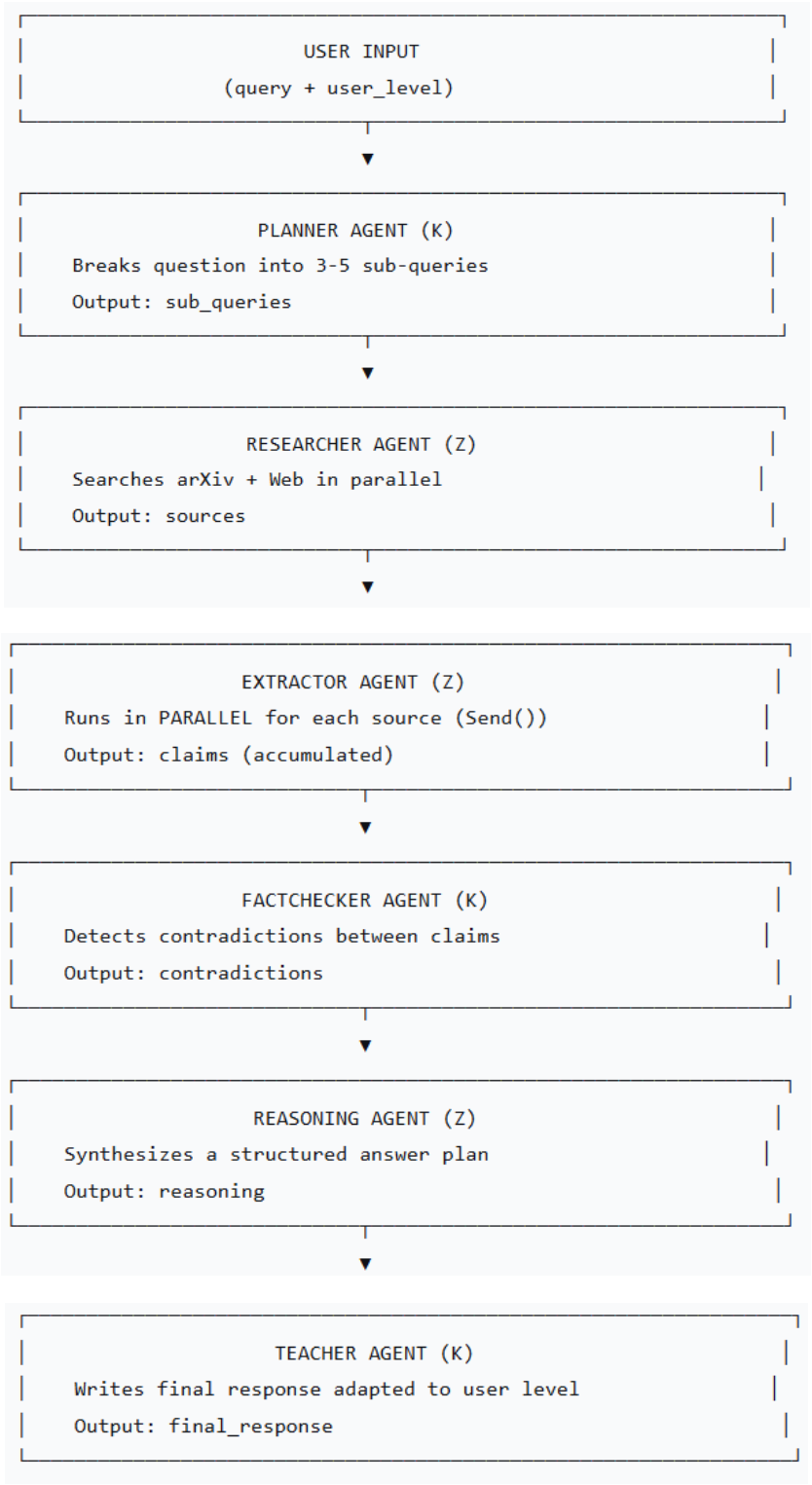
 # --- Errors ---

error: Optional[str]

4. Graph Builder Design

File: src/agents/graph.py

The Pipeline



Key Feature: Parallel Extraction

```python
def create_extraction_jobs(state: GraphState) -> list[Send]:
    """
    Creates one Send() job per source for parallel execution.
    """
    jobs = []
    for source in state["sources"]:
        jobs.append(
            Send("extractor", {"source": source, "session_id": state["session_id"]})
        )
    return jobs
```

Each Send() runs the Extractor node independently on one source. All results are merged back into claims via operator.add.

---

5. Why This Design

Decision	Rationale
Flattened state (not nested)	LangGraph merges partial updates by key. Flattened keys merge cleanly.
`Annotated[list, operator.add]`	Automatically merges results from parallel Extractors.
`Send()` **for extraction**	Runs one Extractor per source in parallel → 40% faster.
SessionSchema fields at top level	Easier for agents to access without nested dict lookups.

---

6. What We Already Have

Component	Status	File
SQLAlchemy models	Done	src/db/models.py
Pydantic schemas	Done	src/schemas/
DuckDuckGo client	Done	src/clients/brave_client.py
arXiv client	Done	src/clients/arxiv_client.py

Component	Status	File
Unit tests	Done	tests/unit/

Sprint 2 builds on all of this.

---

7. Sprint 2 — Full Breakdown

Task Breakdown

Task	Owner	Est.	Deliverable
LangGraph structure	K+Z	4h	state.py, graph.py skeleton
Planner Agent	K	5h	Decompose query → sub-queries
Researcher Agent	Z	5h	Parallel arXiv + web search
FAISS pipeline	K	5h	Embedding, index, save/load
Extractor Agent	Z	6h	Extract claims via Groq LLM
Parallel extraction (Send)	K+Z	3h	Multiple Extractors in parallel
Claims storage	Z	2h	Save claims to SQLite
Streamlit UI (progress)	K	4h	Show agent progress
Streaming responses	K	3h	Stream answers agent-by-agent
Groq wrapper	Z	2h	LLM client + fallback
Unit tests (Planner/Extractor)	Z	3h	Test agents individually

Task	Owner	Est.	Deliverable
Integration tests	K	3h	Test full pipeline

---

8. Next Steps

For Khalil (Sprint 2):

1. Review this design
2. Start building:
    - **Planner Agent** (decomposes questions)
    - **FAISS pipeline** (embeddings)
    - **Streamlit UI** (agent progress)
    - **Integration tests**

For Zeineb (Sprint 2):

1. Build **Researcher Agent** (parallel search)
2. Build **Extractor Agent** (LLM + claims)
3. **Claims storage** in SQLite
4. **Groq wrapper** with fallback
5. **Unit tests** for Planner/Extractor

---

10. Files Created

File	Purpose
src/agents/state.py	GraphState TypedDict
src/agents/graph.py	Graph builder with stub agents
src/agents/__init__.py	Package init

Conclusion

This design is:

- **Simple** — Flat state, clear nodes
- **Fast** — Parallel extraction (40% faster)
- **Testable** — Each agent can be tested independently
- **Scalable** — Easy to add more agents later